

Assignment 1

Video

[CS3358 Assignment1 HunterM](#)

Purpose

Menu based program allows user to choose either stack or queue to test along with which implementation would like to be used. Additional functionality of a postfix expression evaluator using a pointer based stack to handle logic.

Files

- **main.cpp** : Contains main driver for program and testing
- **utils.cpp/h** : Houses Node struct for pointer implementations and various helper functions
- **stack.h** : Declaration of both stack implementations classes and related functions
- **stackArr.cpp** : Actual implementation of the array based stack
- **stackPtr.cpp** : Actual implementation of the pointer based stack
- **queue.h** : Declaration of both queue implementations classes and related functions
- **queueArr.cpp** : Actual implementation of the array based queue
- **queuePtr.cpp** : Actual implementation of the pointer based queue

Structs

- **Node** : Contains an integer value and pointer to another Node

Classes

- **stackArr** : Stack ADT implemented through a dynamic array (LIFO)
- **stackPtr** : Stack ADT implemented through a linked list
- **queueArr** : QueueADT implemented through a dynamic circular array (FIFO)
- **queuePtr** : Queue ADT implemented through a linked list

stackArr Functions

- **stackArr(int iCapacity)** : Constructor needing iCapacity for the size of the array
- **~stackArr()** : Destructor to avoid memory leaks
- **void push(int iNewValue)** : Push element to top of stack

- **bool pop(int& iReturn)** : Pop element at top of stack, returns bool based on if it was successful
- **bool isEmpty()** : Returns bool based on if stack is empty
- **bool isFull()** : Returns bool based on if stack is full
- **int getSize()** : Returns size of the stack
- **int getCapacity()** : Returns capacity of stack

stackPtr Functions

- **stackPtr()** : Constructor which initializes empty linked list
- **~stackPtr()** : Destructor which iterates through list deleting all nodes
- **void push(int iNewValue)** : Push element to top of stack
- **bool pop(int& iReturn)** : Pop element at top of stack, returns bool based on if it was successful
- **bool isEmpty()** : Returns bool based on if stack is empty
- **bool isFull()** : Returns false because limit is memory, assuming it won't grow that large
- **int getSize()** : Returns size of the stack

queueArr Functions

- **queueArr(int iCapacity)** : Constructor needing iCapacity for array size
- **~queueArr()** : Destructor, deletes array
- **void enqueue(int iNewVal)** : Enqueue element
- **bool dequeue(int& iVal)** : Dequeue element, return bool based on completion
- **void peek()** : Print front value
- **bool isEmpty()** : Returns bool based on if queue is empty
- **bool isFull()** : Returns bool based on if queue is full
- **int getSize()** : Returns size of Queue
- **int getCapacity()** : Returns capacity

queuePtr Functions

- **queuePtr()** : Constructor, initialized empty list
- **~queuePtr()** : Destructor, iterates through list and deletes all nodes

- **void enqueue(int iNewVal)** : Enqueue element
- **bool dequeue(int& iVal)** : Dequeue element, return bool based on completion
- **void peek()** : Print front value
- **bool isEmpty()** : Returns bool based on if queue is empty
- **bool isFull()** : Returns false for same reason as stackPtr.isFull()
- **int getSize()** : Returns size of Queue

Miscellaneous Functions

- **void displayMenu(std::string szMenu[], int iLength)** : Prints a string array containing title followed by options.
- **void intInput(int& iUserInput)** : Function used to insure proper int input
- **void postfixExpressionSolver()** : Function containing all logic for the postfix expression solving
- **void arrStackTest(int& iUserInput)** : Function containing all logic for testing the array based stack
- **void ptrStackTest(int& iUserInput)** : Function containing all logic for testing the pointer based stack
- **void arrQueueTest(int& iUserInput)** : Function containing all logic for testing the array based queue
- **void ptrQueueTest(int& iUserInput)** : Function containing all logic for testing the pointer based queue